

SUAVE battery monitoring and adaptation

Shown for the extended BehaviorTree manager (suave_bt_extended), which is the only manager that reacts to the battery. The first page is the full sequence, and the next four pages show each phase with notes on every step; step numbers match across all diagrams. The diagrams show only what the code does today. Missing parts are listed separately at the end and are labelled **G1**, **G2** and so on where the captions refer to them.

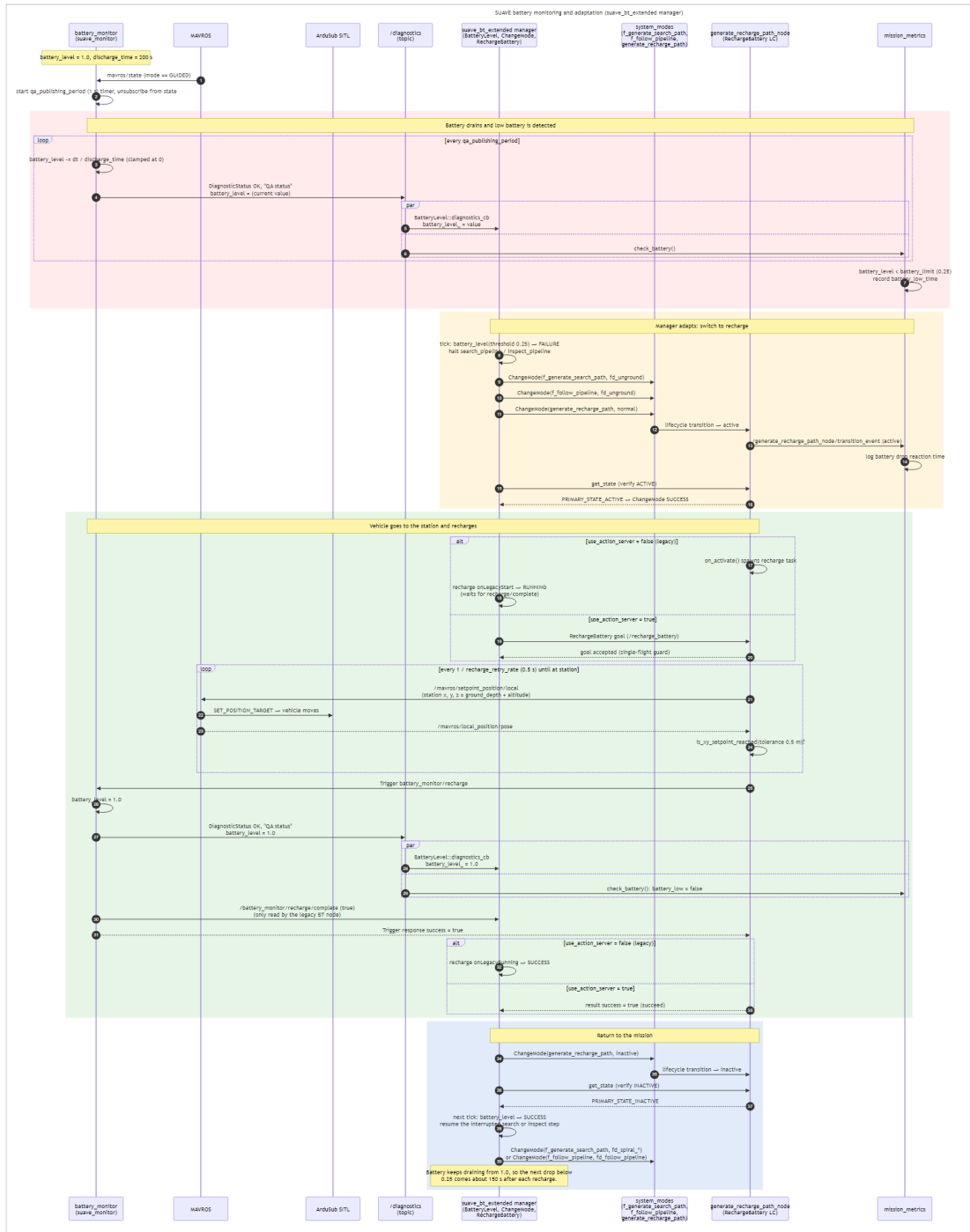


Figure 1. Full sequence. The battery level drains over time and is reported on `/diagnostics` (phase 1). Below 0.25, the manager stops searching or inspecting and activates the recharge node (phase 2). The vehicle goes to the station and calls the monitor's recharge service, which resets the level to 1.0 (phase 3). The manager then deactivates the recharge node and continues the step that was interrupted (phase 4).

Phase 1: battery drains and low battery is detected (steps 1–7)

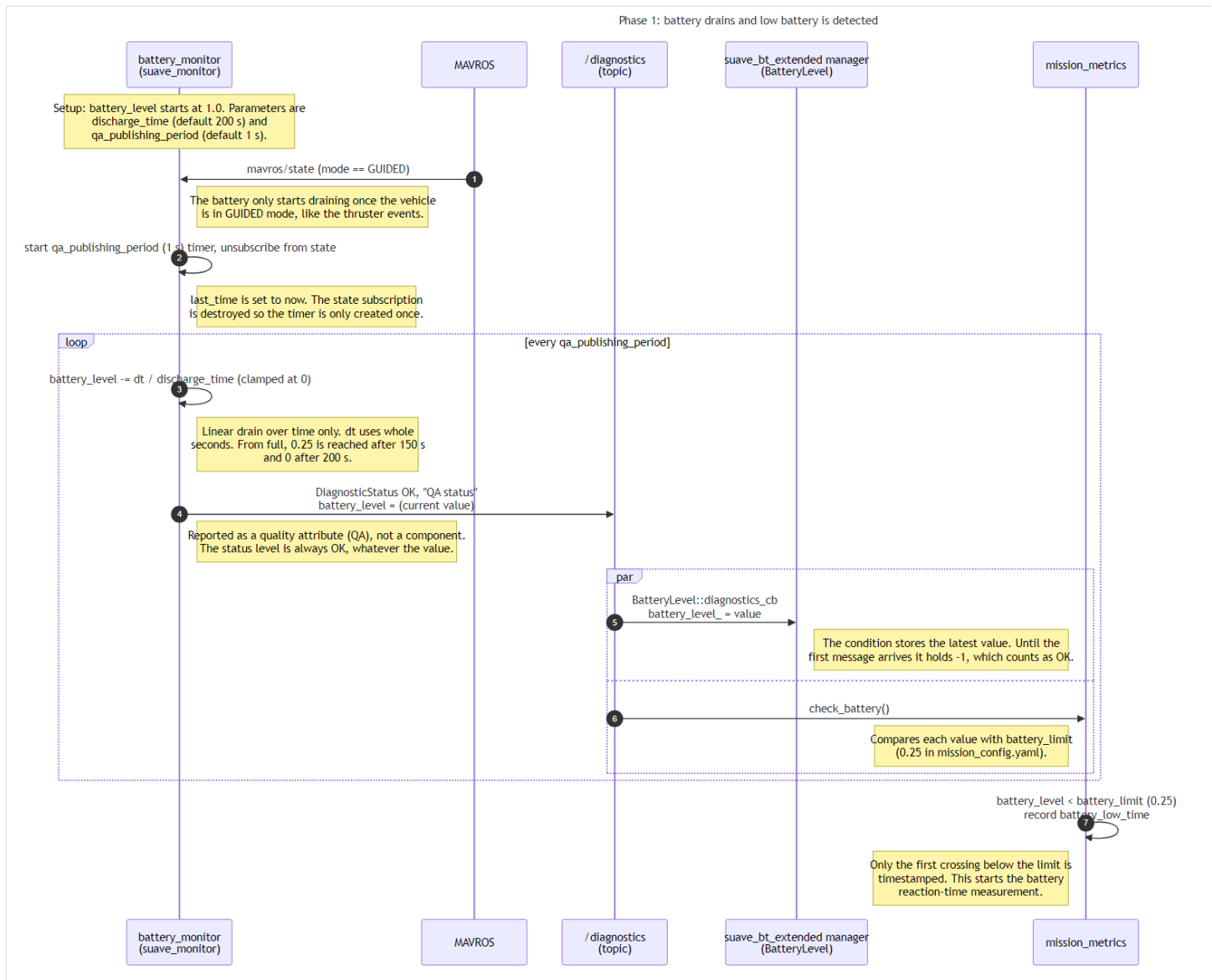


Figure 2. `battery_monitor` simulates a draining battery and reports it as a quality attribute.

- The battery starts draining only after MAVROS reports `GUIDED` mode. From then on, the monitor lowers the level by `dt / discharge_time` once per `qa_publishing_period`. With the defaults (200 s, 1 s), the level reaches 0.25 after 150 s and 0 after 200 s. **G5 G6**
- Each value is published on `/diagnostics` as `battery_level` with the message "QA status". The status level is always OK, even when the battery is low or empty. **G3**
- The BT's `BatteryLevel` condition stores the latest value. `mission_metrics` compares each value with `battery_limit` (0.25). The first time the level falls below the limit, it records the time, which starts the battery reaction-time measurement. **G4**

Phase 2: manager adapts by switching to recharge (steps 8–16)

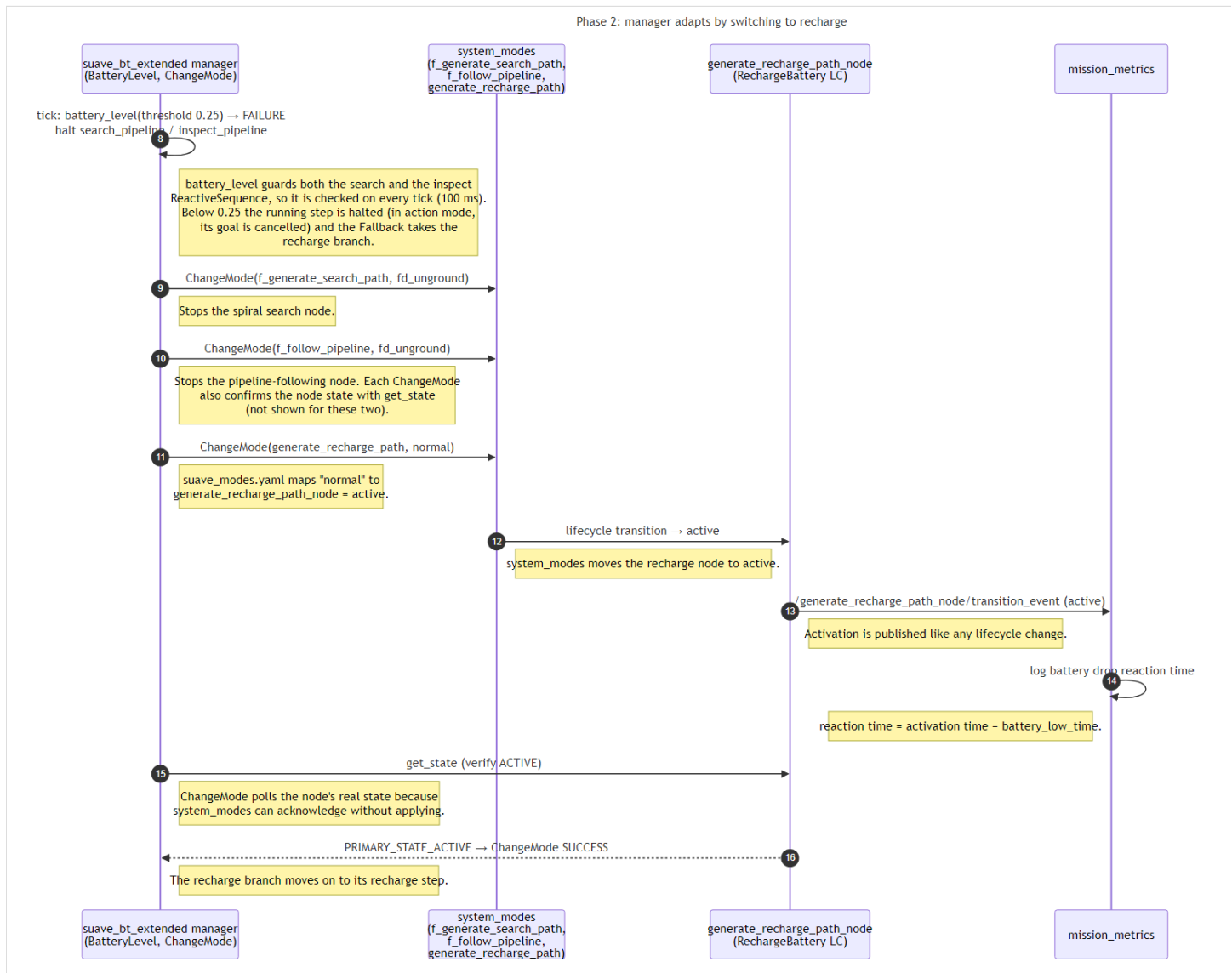


Figure 3. The manager stops the mission work and activates the recharge function through system_modes.

- `battery_level threshold="0.25"` guards both the search and the inspect ReactiveSequence, so it is checked on every tick. When the level drops below 0.25, the running step is halted and the Fallback takes the recharge branch. **G1 G11**
- The recharge branch first sets `f_generate_search_path` and `f_follow_pipeline` to `fd_unground`, which stops both mission nodes. It then sets `generate_recharge_path` to `normal`, which activates `generate_recharge_path_node`.
- As with thrusters, `ChangeMode` confirms each change with `get_state`. The activation shows up on the node's `transition_event`, and `mission_metrics` logs the reaction time (activation time minus low-battery time). **G12**

Phase 3: vehicle goes to the station and recharges (steps 17–33)

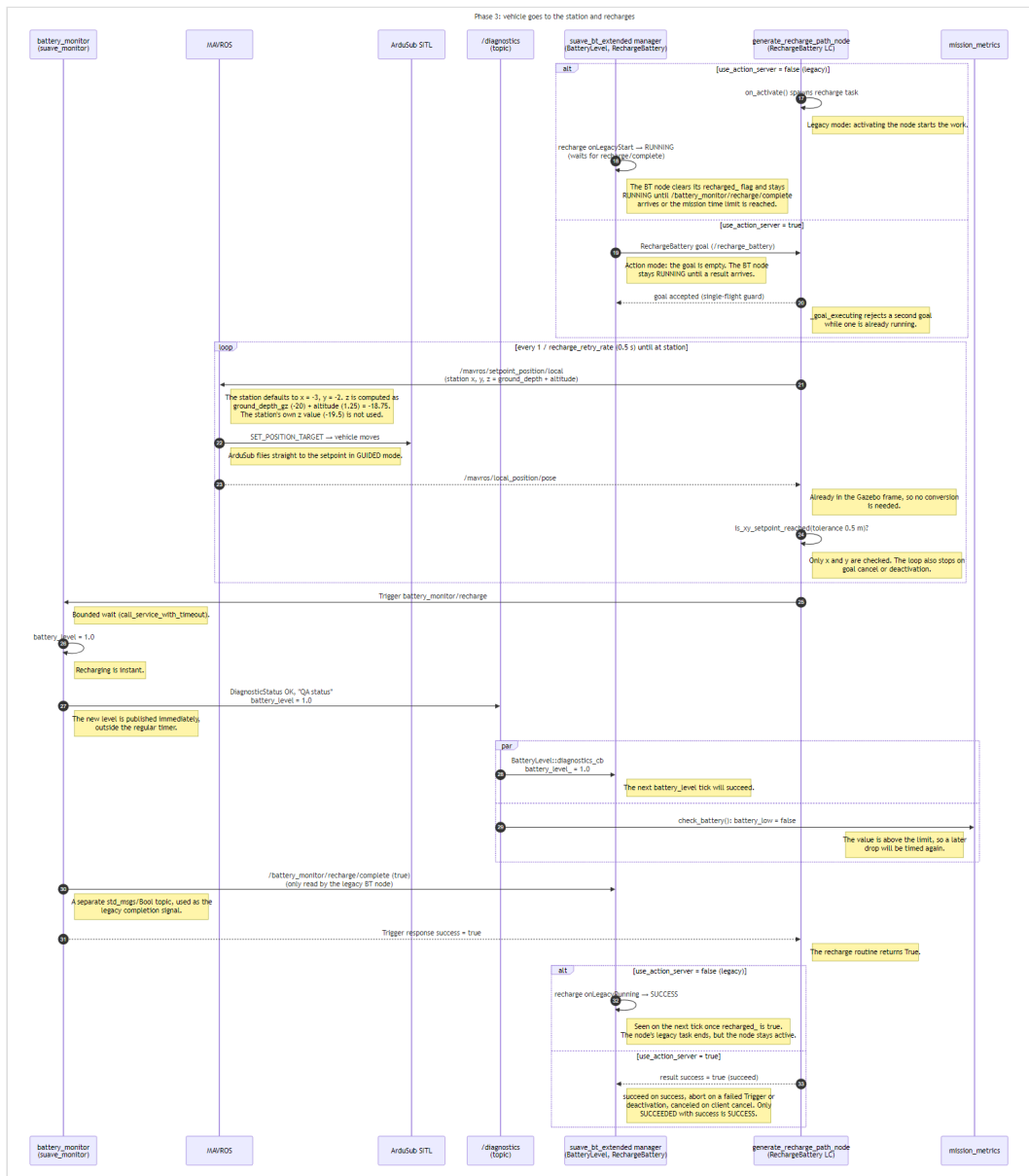


Figure 4. RechargeBattery flies the vehicle to the station and asks the monitor to recharge.

- **How recharge starts.** By default (`use_action_server = false`), activating the node starts recharging, and the BT step stays `RUNNING` until `/battery_monitor/recharge/complete` arrives. In action mode, the BT sends a `RechargeBattery` goal and waits for the result. **G9**
- Every 0.5 s, the node publishes a setpoint at the station's x and y (default -3, -2), at `ground_depth_gz + altitude` (-18.75). It stops when the vehicle's x and y are within 0.5 m of the station. **G8**
- The node then calls the `battery_monitor/recharge` Trigger service. The monitor sets the level to 1.0 straight away, publishes it on `/diagnostics` and on `recharge/complete`, and replies with success. **G6 G7**
- The BT condition and the metrics both see 1.0, so the low-battery state is cleared. In action mode, the result is `succeed` on success, `abort` on a failed Trigger or deactivation, and `cancel` on a client cancel.

Phase 4: return to the mission (steps 34–39)

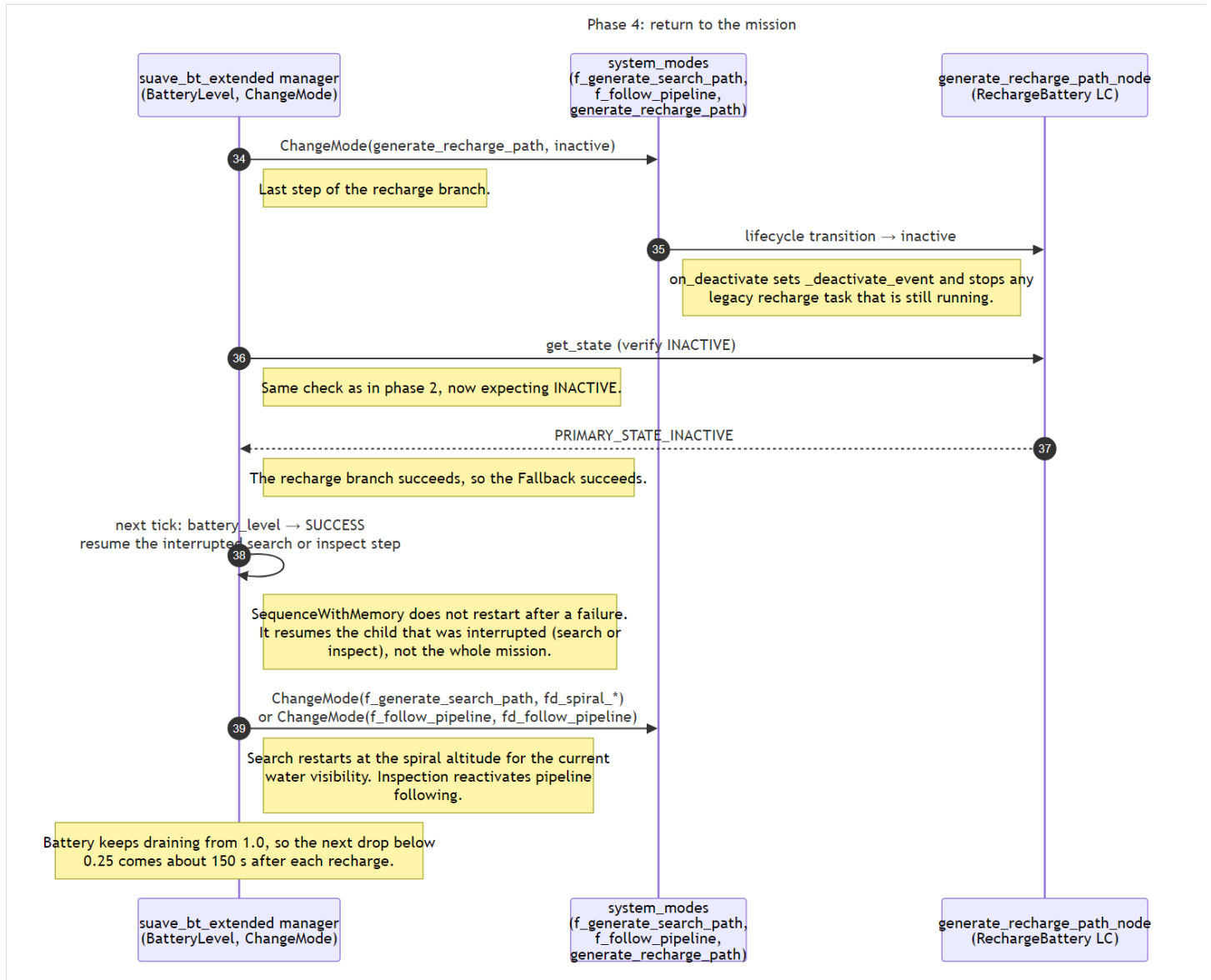


Figure 5. The manager deactivates the recharge node and continues where the mission left off.

- The recharge branch ends by setting `generate_recharge_path` to `inactive`. `system_modes` deactivates the node, which stops any legacy recharge task still running. `ChangeMode` confirms the node is `INACTIVE`. **G10**
- On the next tick, `battery_level` succeeds. Because `SequenceWithMemory` doesn't restart after a failure, the tree continues the step that was interrupted. A search goes back to the spiral altitude for the current water visibility; an inspection reactivates pipeline following.
- The battery starts draining again from 1.0, so the next recharge is due about 150 s later.

Missing parts

These are gaps found by reading the current code. None of them are drawn in the diagrams above.

Adaptation coverage

G1. Only one manager handles the battery. Only `suave_bt_extended` (`bts/suave_extended.xml`) reacts to the battery. The default `bt_executable` is `suave_bt`, whose tree has no battery nodes, and the none and random managers ignore the battery too. Metacontrol's `suave.owl` defines a `battery_level` quality attribute, but no function design uses it, there's no recharge function, and the task bridges have no recharge task. With the default launch settings, the battery drains to 0 and nothing reacts.

`suave_bringup/launch/mission.launch.py`, `suave_managing/suave_bt/bts/`, `suave_managing/suave_metacontrol/config/suave.owl`

G2. An empty battery has no effect. At 0 the monitor simply holds the level at 0, and the vehicle keeps working normally. A thruster failure really disables a motor, but the battery level never affects the vehicle. The only low-battery abort is `battery_constraint` in `inspection_mission.py`. It is off by default, and that mission node isn't launched with the BT manager.

`suave_monitor/suave_monitor/battery_monitor.py`, `suave_missions/suave_missions/inspection_mission.py`

Monitor and diagnostics

G3. Diagnostics never warn. The battery is always reported with level OK, even when it's low or empty, and the monitor doesn't know the low-battery threshold. Generic `/diagnostics` tools therefore can't tell that the battery is low.

G4. The threshold is set in several places. The BT XML hard-codes 0.25 twice, `mission_metrics` uses `battery_limit` 0.25, and the mission node uses `battery_constraint_value` 0.2 (a launch default). Changing one of them won't change the others.

G5. Battery settings can't be configured per mission. The mission configs have no `/battery_monitor` section, so `discharge_time` is always 200 s and `qa_publishing_period` always 1 s. The `qa_publishing_period` entries in those files belong to the water visibility observer. `suave_runner` has no battery disturbance settings either: there's no equivalent of `thruster_events` and no randomised drain rate.

G6. The battery model is very simple. The drain depends only on time, not on load or thruster use. It works in whole seconds (`to_msg().sec`) and keeps draining on the way to and at the station. Recharging is instant, and there is no "charging" state.

G7. The recharge interface is loose. Anything can call `battery_monitor/recharge`, and the monitor doesn't check that the vehicle is at the station. The completion signal goes out on its own topic (`/battery_monitor/recharge/complete`), outside the standard managing interfaces (`/diagnostics`, `/task/*`, `change_mode`).

Recharge behaviour

G8. The station and approach are fixed. The station position, altitude and ground depth are only node defaults. The launch file passes nothing but `use_action_server`, so they can't be set from the mission config. The station's z value (−19.5) is never used. Arrival is checked in x and y only, so depth isn't verified before recharging, and the vehicle flies a straight line with no path planning.

`suave/suave/recharge_battery_lc.py`, `suave/launch/suave.launch.py`

G9. Legacy mode never reports a failed recharge. If the Trigger call fails or times out, the node logs an error and its task ends without retrying. The BT step is waiting for `recharge/complete`, so it stays RUNNING until the mission time limit. Action mode does report `abort`.

G10. Stopping the recharge isn't fully handled. `on_deactivate` asks the legacy task to stop but doesn't wait for it, whereas `RecoverThrustersLC` waits up to 2 s. `on_cleanup` and `on_shutdown` wait for the task with no timeout.

Manager, metrics and tests

G11. The BatteryLevel condition has weak spots. Before the first message arrives, the level is unknown (−1) and counts as OK. A `battery_level` value that isn't a number makes `std::stof` throw, which would crash the BT.

G12. Battery metrics are limited. Only the reaction time is recorded. There's nothing for the number of recharges, time spent recharging, the lowest level reached, or running out of battery. The `battery_low` reset in `generate_recharge_path_transition_cb` is commented out, so another reaction time is logged every time the node is activated while the battery is still low.

G13. Tests are missing. There are no unit tests for `battery_monitor` (`suave_monitor/test` only covers the water visibility observer) or for the `BatteryLevel` condition. `RechargeBattery` LC and the BT recharge action client do have tests.